Terraform CI/CD Demo

Building Infrastructure as Code from Scratch

A Beginner's Guide to:

- Infrastructure as Code (IaC)
- Terraform
- AWS Services
- GitHub Actions CI/CD

📚 What We'll Build

By the end, you'll have:

- ✅ Automated infrastructure deployment
- ✅ Three environments (dev, staging, production)
- ✅ S3 buckets connected to Lambda functions
- ✅ Terraform state stored remotely
- ✅ GitHub Actions automating everything

And you'll understand every piece!

🎯 Learning Path

```
Part 1: Core Concepts → Part 2: Basic Setup → Part 3: Modules
                                    ↓
Part 6: CI/CD ← Part 5: Environments ← Part 4: Remote State
```

We'll build incrementally, testing at each step.

Part 1: Core Concepts

What is Infrastructure as Code?

Traditional Way:

1. Log into AWS Console
2. Click buttons to create S3 bucket
3. Click more buttons to create Lambda
4. Write down what you did (maybe)
5. Repeat manually for staging and production

Problem: Slow, error-prone, not reproducible

Part 1: Core Concepts

Infrastructure as Code (IaC)

IaC Way:

1. Write code describing your infrastructure
2. Run `terraform apply`
3. Infrastructure is created automatically
4. Code is version controlled
5. Same code = same infrastructure every time

Benefits: Fast, reliable, reproducible

Part 1: What is Terraform?

Terraform is a tool that:

- Reads code describing infrastructure
- Talks to cloud providers (AWS, Azure, GCP)
- Creates, updates, or destroys resources
- Tracks what exists (state management)

Think of it as a translator between your code and AWS.

Part 1: What We're Building

```
┌──────────────┐
│   S3 Bucket  │ ──(upload file)──┐
└──────────────┘                  ▼
                        ┌──────────────────┐
                        │ Lambda Function  │
                        │  (processes it)  │
                        └──────────────────┘
```

Purpose: When you upload a file to S3, Lambda automatically processes it.

Real-world use: Image resizing, data processing, file validation, etc.

Part 1: AWS Services We'll Use

S3 (Simple Storage Service)

- Like Dropbox/Google Drive
- Stores files in "buckets"
- Cheap, reliable storage

Lambda

- Runs code without managing servers
- Triggered by events (like S3 uploads)
- Pay only when code runs

Part 1: AWS Services (Continued)

DynamoDB

- Database service
- We'll use it for "state locking"
- Prevents conflicts when multiple people deploy

IAM (Identity and Access Management)

- Controls who/what can access AWS resources
- We'll create roles for Lambda

Part 2: Setting Up Your Environment

Prerequisites

Install these tools:

1. **Terraform** - terraform.io/downloads

```
terraform version  # Should show >= 1.0
```

2. **AWS CLI** - aws.amazon.com/cli

```
aws --version
```

3. **Git** - git-scm.com

```
git --version
```

Part 2: AWS Credentials

You'll need:

- AWS Access Key ID
- AWS Secret Access Key
- AWS Session Token (if using temporary credentials)

Set them up:

```
export AWS_ACCESS_KEY_ID="your-access-key"
export AWS_SECRET_ACCESS_KEY="your-secret-key"
export AWS_SESSION_TOKEN="your-token"
export AWS_REGION="us-east-1"
```

Test it:

```
aws sts get-caller-identity
```

Part 2: Project Structure

```
terraform-cicd-demo/
├── backend-setup.tf       ← Creates state storage (do first)
├── modules/               ← Reusable components
│   ├── s3_bucket/
│   └── lambda_function/
├── environments/          ← Where we deploy
│   ├── main.tf           ← Shared template
│   ├── dev.tfvars        ← Dev config
│   ├── staging.tfvars    ← Staging config
│   └── production.tfvars ← Production config
└── .github/workflows/    ← CI/CD automation
```

Part 3: Understanding Modules

What is a Module?

A module is a reusable piece of infrastructure.

Instead of writing the same S3 bucket code 3 times (dev, staging, production), we write it once in a module and reuse it.

```
┌──────────────────┐
│ S3 Module        │  ──uses──┐
│ (write once)     │          │
└──────────────────┘          ▼
                  ┌──────────────────────┐
                  │ Dev   Staging   Prod │
                  └──────────────────────┘
```

Part 3: S3 Bucket Module

File: `modules/s3_bucket/main.tf`

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  tags   = var.tags
}

resource "aws_s3_bucket_encryption" "this" {
  bucket = aws_s3_bucket.this.id
  # Encrypts everything automatically
}

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id
  # Keeps old versions of files
}
```

Part 3: S3 Module - Variables

File: `modules/s3_bucket/variables.tf`

```
variable "bucket_name" {
  description = "Name of the S3 bucket"
  type        = string
}

variable "enable_versioning" {
  description = "Enable versioning?"
  type        = bool
}

variable "tags" {
  description = "Tags to apply"
  type        = map(string)
}
```


Variables are like function parameters

Part 3: S3 Module - Outputs

File: `modules/s3_bucket/outputs.tf`

```
output "bucket_id" {
  value = aws_s3_bucket.this.id
}

output "bucket_arn" {
  value = aws_s3_bucket.this.arn
}
```

Outputs are like function return values.

Other parts of your code can reference these.

Part 3: Lambda Module

File: `modules/lambda_function/main.tf`

```
resource "aws_lambda_function" "this" {
  filename      = "lambda.zip"
  function_name = var.function_name
  role          = aws_iam_role.lambda_role.arn
  handler       = "lambda_function.lambda_handler"
  runtime       = "python3.11"
}
```

This creates a Lambda function that can run Python code.

Part 3: Lambda Code

File: `modules/lambda_function/lambda_function.py`

```python
import json
import os

def lambda_handler(event, context):
    # This runs when a file is uploaded to S3
    environment = os.environ.get('ENVIRONMENT')

    for record in event['Records']:
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']

        print(f"File uploaded to {bucket}/{key}")
        print(f"Environment: {environment}")

    return {'statusCode': 200}
```

Part 4: Remote State

Why Do We Need It?

Terraform State tracks what infrastructure exists.

Problem with local state:

- Only on your computer
- Can't collaborate with team
- No backup if computer crashes

Solution: Remote State

- Stored in S3 (cloud)
- Everyone can access it
- Locked with DynamoDB (prevents conflicts)

Part 4: Backend Setup

File: `backend-setup.tf`

```
resource "aws_s3_bucket" "terraform_state" {
  bucket = "terraform-cicd-demo-state-bucket"
  # Stores terraform.tfstate files
}

resource "aws_dynamodb_table" "terraform_locks" {
  name     = "terraform-cicd-demo-locks"
  hash_key = "LockID"
  # Prevents two people deploying at once
}
```

Run this FIRST, before anything else:

```
terraform init
terraform apply
```

Part 5: Environments

What are Environments?

Environment = A separate copy of your infrastructure

Why 3 environments?

1. **Dev** (development)
    - For testing locally
    - Break things safely
    - Fast iteration
2. **Staging** (pre-production)
    - Test before going live
    - Identical to production

Part 5: Environment Configuration

Templated Approach

One template + Different configurations = Different environments

```
┌──────────────────┐
│   main.tf        │   ← Shared template
│  (write once)    │
└──────────────────┘
        │
    ┌───┴───┬───┬───┐
    ▼       ▼   ▼   ▼
dev.tfvars staging  production
           .tfvars   .tfvars
```

Part 5: Main Template

File: `environments/main.tf`

```
module "app_bucket" {
  source = "../modules/s3_bucket"

  bucket_name       = "app-${var.environment}-${random_id.suffix.hex}"
  enable_versioning = var.enable_bucket_versioning
  tags              = { Environment = var.environment }
}

module "s3_processor" {
  source = "../modules/lambda_function"

  function_name = "processor-${var.environment}"
  environment   = var.environment
}
```

Part 5: Dev Configuration

File: `environments/dev.tfvars`

```
environment               = "dev"
enable_bucket_versioning = false  # Save money
lambda_log_level        = "DEBUG" # More logging
```

Part 5: Staging Configuration

File: `environments/staging.tfvars`

```
environment               = "staging"
enable_bucket_versioning = false  # Still saving money
lambda_log_level         = "INFO"  # Normal logging
```

Part 5: Production Configuration

File: `environments/production.tfvars`

```
environment               = "production"
enable_bucket_versioning = true   # Keep backups!
lambda_log_level        = "WARN"  # Only important stuff
```

Part 5: Environment Variables

File: `environments/variables.tf`

```
variable "environment" {
  description = "Environment name"
  type        = string
  default     = "dev"  # Defaults to dev for local work
}

variable "enable_bucket_versioning" {
  type    = bool
  default = false
}

variable "lambda_log_level" {
  type    = string
  default = "INFO"
}
```

Part 6: GitHub Actions CI/CD

What is CI/CD?

CI = Continuous Integration

- Automatically test code when you push
- Find problems early

CD = Continuous Deployment

- Automatically deploy when you merge
- No manual steps

Benefits:

- Less human error
- Faster deployments

Part 6: The Workflow

```
Developer          GitHub              AWS
    │                 │                  │
    ├─ Push code ─────▶│                  │
    │                 │                  │
    │                 ├─ Run tests       │
    │                 ├─ Plan infra      │
    │                 │                  │
    ├─ Merge PR ──────▶│                  │
    │                 │                  │
    │                 ├─ Deploy ─────────▶│
    │                 │                  │
    │◀─ Get summary ──┤◀── Confirm ──────┤
```

Part 6: Workflow File

`.github/workflows/terraform-cicd.yml`

Structure:

```yaml
name: Terraform CI/CD

on: [pull_request, push]  # When to run

jobs:
  staging:    # Job 1: Handle staging
  production: # Job 2: Handle production
```

Each job:

1. Plans on PR (shows what will change)
2. Applies on merge (actually deploys)

Part 6: Staging Job

```
staging:
  if: github.ref == 'refs/heads/staging'

  steps:
    - Checkout code
    - Setup Terraform
    - Configure AWS credentials
    - Run: terraform init
    - Run: terraform plan -var-file="staging.tfvars"
    - If PR: Post plan as comment
    - If merge: terraform apply -var-file="staging.tfvars"
    - Show deployment summary
```

Part 6: Production Job

```yaml
production:
  if: github.ref == 'refs/heads/main'

  steps:
    - Same as staging...
    - But uses: production.tfvars
    - And adds warning: "DEPLOYING TO PRODUCTION"
```

🛠️ Implementation Guide

Step 1: Create Backend (15 minutes)

```
# 1. Create project directory
mkdir terraform-cicd-demo
cd terraform-cicd-demo

# 2. Create backend-setup.tf
# (Copy from project)

# 3. Set up AWS credentials
export AWS_ACCESS_KEY_ID="..."
export AWS_SECRET_ACCESS_KEY="..."

# 4. Create backend
terraform init
terraform apply

# ✅ You now have remote state storage!
```

🛠️ Implementation Guide

Step 2: Create Modules (20 minutes)

```
# 1. Create module directories
mkdir -p modules/s3_bucket
mkdir -p modules/lambda_function

# 2. Create module files
# - modules/s3_bucket/main.tf
# - modules/s3_bucket/variables.tf
# - modules/s3_bucket/outputs.tf
# - modules/lambda_function/main.tf
# - modules/lambda_function/variables.tf
# - modules/lambda_function/outputs.tf
# - modules/lambda_function/lambda_function.py

# ✅ You have reusable components!
```

🛠️ Implementation Guide

Step 3: Create Environment Template (15 minutes)

```
# 1. Create environments directory
mkdir environments
cd environments

# 2. Create template files
# - main.tf (uses the modules)
# - variables.tf (defines inputs)
# - outputs.tf (exposes values)

# 3. Create .tfvars files
# - dev.tfvars
# - staging.tfvars
# - production.tfvars

# ✅ You have environment configs!
```

🛠️ Implementation Guide

Step 4: Test Locally (10 minutes)

```
cd environments

# Initialize
terraform init

# Plan (defaults to dev)
terraform plan

# Apply
terraform apply

# Test
BUCKET=$(terraform output -raw bucket_name)
echo "test" > test.txt
aws s3 cp test.txt s3://$BUCKET/uploads/test.txt

# Check Lambda logs
```

🛠️ Implementation Guide

Step 5: Set Up GitHub (15 minutes)

```
# 1. Create GitHub repo
# Go to github.com → New repository

# 2. Initialize git
git init
git add .
git commit -m "Initial commit"

# 3. Push to GitHub
git remote add origin https://github.com/YOUR_USER/terraform-cicd-demo.git
git branch -M main
git branch staging
git push -u origin main
git push -u origin staging

# ✅ Code is on GitHub!
```

🛠️ Implementation Guide

Step 6: Configure Secrets (5 minutes)

On GitHub:

1. Go to Settings
2. Click "Secrets and variables"
3. Click "Actions"
4. Add secrets:
    - `AWS_ACCESS_KEY_ID`
    - `AWS_SECRET_ACCESS_KEY`
    - `AWS_SESSION_TOKEN`

✅ GitHub can access AWS!

🛠️ Implementation Guide

Step 7: Create Workflow (10 minutes)

```
# 1. Create workflow directory
mkdir -p .github/workflows

# 2. Create workflow file
# .github/workflows/terraform-cicd.yml
# (Copy from project)

# 3. Push to GitHub
git add .github/workflows
git commit -m "Add CI/CD workflow"
git push origin main
git push origin staging

# ✅ CI/CD is active!
```

🛠️ Implementation Guide

Step 8: Test CI/CD (15 minutes)

```
# 1. Create feature branch
git checkout staging
git checkout -b test-cicd

# 2. Make a change
echo "# Test" >> README.md
git add README.md
git commit -m "Test CI/CD"

# 3. Push and create PR
git push origin test-cicd
# Go to GitHub → Create PR to staging

# 4. Watch GitHub Actions run!
# - Shows terraform plan
# - Posts comment on PR

# 5. Merge PR
```

🎓 Key Concepts Review

Infrastructure as Code

→ Infrastructure defined in code files

Terraform

→ Tool that reads IaC and creates infrastructure

Modules

→ Reusable pieces of infrastructure

State

→ What infrastructure currently exists

Remote Backend

→ Storing state in the cloud (S3 + DynamoDB)

🎓 Key Concepts Review (Cont'd)

Environments

→ Separate copies of infrastructure (dev, staging, prod)

tfvars Files

→ Configuration specific to each environment

CI/CD

→ Automated testing and deployment

GitHub Actions

→ Tool that runs workflows automatically

🔍 Understanding the Flow

Local Development:

```
You → terraform apply → AWS (dev environment)
```

CI/CD Flow:

```
You → GitHub (PR) → Plan runs → Review
                        ↓
You → Merge PR → Apply runs → AWS (staging/prod)
```

Why this is powerful:

- Changes are reviewed before deploying
- Deployments are automated and consistent
- History is tracked in git

💡 Best Practices

1. Always test locally first

```
terraform plan  # See what will change
terraform apply # Make the changes
```

2. Use meaningful commit messages

```
git commit -m "Add S3 versioning to production"
```

3. Review terraform plans carefully

- Check what's being created
- Check what's being deleted
- Verify configurations

💡 Best Practices (Cont'd)

4. Deploy incrementally

- Dev → Staging → Production
- Test at each stage

5. Use tags for organization

```
tags = {
  Environment = "staging"
  Project     = "demo"
  ManagedBy   = "Terraform"
}
```

6. Keep state files safe

- Never delete the state bucket
- Never manually edit state files

🐛 Troubleshooting Common Issues

Problem: AWS credentials expired

```
# Get fresh credentials
export AWS_ACCESS_KEY_ID="new-key"
export AWS_SECRET_ACCESS_KEY="new-secret"

# Verify
aws sts get-caller-identity
```

Problem: State locked

```
# Check locks
aws dynamodb scan --table-name terraform-cicd-demo-locks

# Force unlock (use lock ID from error)
terraform force-unlock <LOCK_ID>
```

🐛 Troubleshooting (Cont'd)

Problem: Bucket name already exists

- Bucket names are globally unique
- Random suffix is added automatically
- If still fails, change bucket name prefix

Problem: Permission denied

- Check AWS credentials
- Verify IAM permissions
- Ensure role has required access

Problem: GitHub Actions failing

- Check GitHub secrets are set

📊 Cost Considerations

AWS Free Tier includes:

- 5 GB S3 storage
- 20,000 GET requests
- 2,000 PUT requests
- 1M Lambda requests/month
- 400,000 GB-seconds of compute

This project uses:

- ~0.1 GB storage (minimal)
- ~100 requests/month (testing)
- ~10 Lambda invocations/month

🚀 Next Steps

After mastering this project:

1. **Add more services**
    - API Gateway
    - DynamoDB tables
    - SNS notifications
2. **Add monitoring**
    - CloudWatch dashboards
    - Alarms
    - Cost alerts
3. **Add testing**

🚀 Next Steps (Cont'd)

4. **Improve security**
    - Enable MFA
    - Use least-privilege IAM
    - Enable CloudTrail
5. **Add more environments**
    - QA environment
    - Demo environment
    - Training environment
6. **Explore advanced features**
    - Terraform workspaces

📚 Additional Resources

Documentation:

- Terraform Docs
- AWS Terraform Provider
- GitHub Actions Docs

Project Files:

- README.md - Complete documentation
- CI_CD_GUIDE.md - CI/CD explanation
- LOCAL_DEVELOPMENT.md - Local workflow

Learning:

- Terraform tutorials on HashiCorp Learn

🎯 Summary

What You Learned:

- ✅ Infrastructure as Code concepts
- ✅ How Terraform works
- ✅ Creating reusable modules
- ✅ Managing multiple environments
- ✅ Remote state management
- ✅ CI/CD with GitHub Actions
- ✅ AWS services (S3, Lambda, DynamoDB, IAM)
- ✅ Best practices and troubleshooting

What You Built:

- ✅ Production-ready infrastructure
- ✅ Automated deployment pipeline

🎉 Congratulations!

You now have a **production-ready** infrastructure project with:

- Automated deployments
- Multiple environments
- Remote state management
- CI/CD pipeline
- Best practices implemented

This is a strong foundation for any cloud infrastructure project!

📞 Questions?

Common questions:

Q: Can I use this in production?

A: Yes! But add monitoring, alerts, and backups first.

Q: How do I add more AWS services?

A: Create new modules similar to s3_bucket and lambda_function.

Q: Can I use Azure or GCP instead?

A: Yes! Change the provider and adapt the resources.

Q: How do I collaborate with a team?

A: Remote state (S3) already enables team collaboration!

🎓 Final Tips for Beginners

1. **Start small** - Deploy dev first, then expand
2. **Read error messages** - They usually tell you exactly what's wrong
3. **Use terraform plan** - Always check before applying
4. **Keep learning** - Terraform and cloud tech evolve constantly
5. **Ask for help** - Communities are friendly and helpful

Remember: Everyone starts as a beginner.

Keep practicing!

🔗 Get Started Now!

1. **Clone the repo:**

```
git clone https://github.com/bhydemi/terraform-cicd-demo.git
```

2. **Follow the implementation guide** (Slides 58-65)

3. **Join the DevOps community** and share your progress!

Happy building! 🚀

Thank You!

Repository:

https://github.com/bhydemi/terraform-cicd-demo

Documentation:

- README.md
- CI_CD_GUIDE.md
- LOCAL_DEVELOPMENT.md

Questions? Feedback?

Open an issue on GitHub!